GRADE XII · COMPUTER SCIENCE

Lecture Notes

A working set of notes for the six units: how people and computers meet, how algorithms and data structures are judged, how Python stores and files data, how numbers become decisions, how intelligent systems and security affect daily life, and how a small local problem becomes a tested digital product.

- Inline SVG diagrams
- MathJax formulas
- Flexbox page layout

1 **Human–Computer Interaction**
Usability, accessibility, Figma, testing

2 **Computational Thinking**
Correctness, Big O, linear and non-linear structures

3 **Programming Fundamentals**
Lists, tuples, sets, dictionaries, functions, files

4 **Data and Analysis**
SQLite, Pandas, charts, centre and spread

5 **Applications and Impacts**
Neural networks, protection, equity

6 **Digital Entrepreneurship**
Prototype, MVP, beachhead market

HOW TO USE THESE NOTES

Read a section, redraw its diagram from memory, then redo the worked numbers without looking. The short answers at the end of each unit are written in the form an examiner can mark.

Student revision booklet. Original notes on the Grade XII syllabus. Diagrams and wording are drawn for study, not copied from the textbook pages. Formulas are set with MathJax and stored in the PDF as SVG.

How the booklet is built

MAP

What to memorise

Definitions in one sentence, the name of each diagram, and the formula with what every symbol means. Boards reward a clean definition plus one local example.

What to be able to do

Fill a trace table, choose a data structure, slice a list, compute mean and variance, name a threat with its mitigation, and plan an MVP for one small user group.

What the symbols mean

Inline mathematics such as $O(n)$ sits in the sentence. Display mathematics is set on its own line. A gold bar marks a result worth copying into a revision card.

Unit 1 · Human–Computer Interaction	Usability and Figma
Unit 2 · Computational Thinking and Algorithms	Trace tables and Big O
Unit 3 · Programming Fundamentals	Python collections and files
Unit 4 · Data and Analysis	Databases, charts, statistics
Unit 5 · Applications and Impacts of Computing	Networks, safety, equity
Unit 6 · Entrepreneurship in the Digital Age	Prototype to first users
Quick key	Objective answers

1 · Human–Computer Interaction

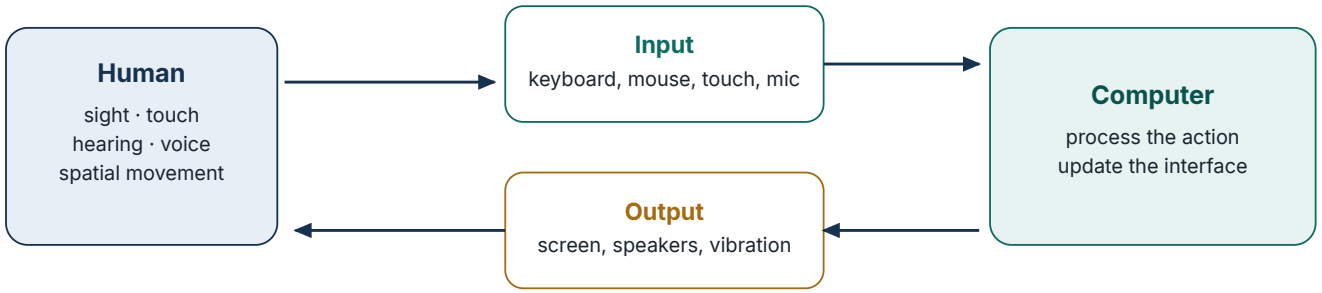
UNIT 1

Human–Computer Interaction (HCI) is the design of technology so that people can finish real tasks with little confusion. The same field is also called CHI (Computer–Human Interface) and MMI (Man–Machine Interaction). The unit joins three ideas: the **user interface** (what is on the screen), the **user experience** (how the whole task feels), and **usability** (whether the task is effective, efficient, and satisfying).

- Explain fundamentals and importance of HCI
- Apply accessibility: contrast, readability, colour, navigation
- Analyse user needs before drawing an interface
- Name common HCI problems and fixes
- Design and prototype in Figma
- Test and evaluate with real measures

1.1 The interaction loop

People and computers meet through the senses. A person acts; a device captures that action; the system computes; an output returns; the person decides what to do next. That return signal is feedback.



Feedback closes the loop: the person sees, hears, or feels the result and acts again.

Figure 1.1. The HCI model. Input carries intention inward. Output and feedback carry the system's answer outward.

Traditional interaction

Everyday devices and explicit controls. On a computer, the mouse and keyboard go in, and the monitor and speakers come out. On a phone, taps and swipes go in, and the screen, sound, or a short vibration comes out.

Natural interaction

The device reads behaviour that already feels human. Voice: "play the lesson." Gesture and body: a camera watches a hand wave or a face to unlock a phone. The person is not hunting for a command name.

1.2 Where HCI shows up

An interface is a human-centred product, not only a technical one. The same design questions appear in healthcare (patient portals, telemedicine, assistive tools), banking (a clear and secure transfer), education (learning platforms and exam portals), and communication (social and collaboration apps).

1.3 Seven parts of an HCI situation

User

End users operate the system, from novice to expert. **Stakeholders** care about the outcome: a client, a manager, an administrator.

Computer system

The platform that joins input, processing, output, and feedback. Hardware and software together are the other side of the conversation.

User interaction

The act that turns intention into a response: click, tap, type, swipe, or speak.

Task

The goal-sized action: book a ticket, search a product, mark attendance. Judge it by complexity, the skill it needs, and the time it should take.

Interface

The medium. GUI, command line, touch, voice, or a natural interface built from gesture and space.

Environment and feedback

The physical and social setting of use. Feedback is the system's confirmation ("Payment successful", a vibration) or the user's own signal (a rating, a spoken correction).

1.4 Styles of interaction

Style	What the user does	Example
Direct manipulation	Handles screen objects as if they were physical	Drag a file, pinch to zoom
Command-based	Types a precise instruction	`dir`, `cd ..`
Menu-driven	Picks from a prepared list	Start test, check result, download certificate
Form fill	Enters structured fields	Admission form, ticket booking
Conversational / voice	Uses ordinary language	A chatbot, "call Aslam"
Gesture, biometric, haptic	Moves, presents a body feature, or feels a tap	VR hand, fingerprint, wrist tap
Context-aware	Lets the system sense the situation	Lights on entry, a route that follows traffic
Multimodal	Combines two or more of the above	Speak a query, then tap a result

1.5 Families of interface

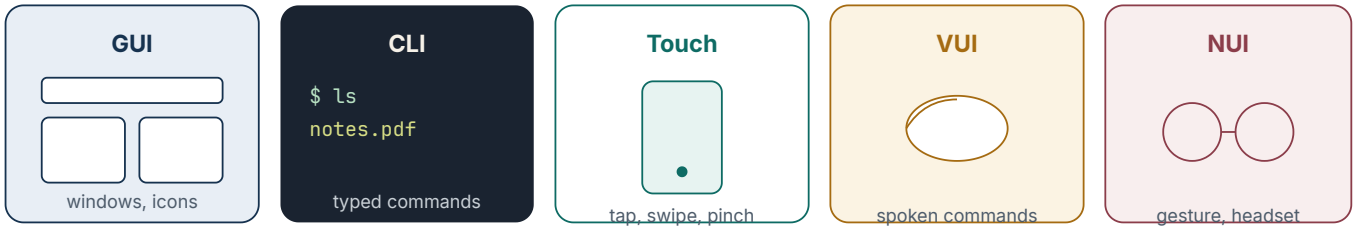


Figure 1.2. Five interface families. A product often mixes them: a phone is touch first, and also voice and biometric.

1.6 Why HCI matters

Good design turns a complicated machine into an everyday tool. A clear phone screen lets a student reach a lesson. A crowded medical screen, or a site buried in advertisements, creates error and delay. The practical consequences are:

1. **Productivity.** Fewer steps and fewer mistakes.
2. **Inclusivity.** Screen readers, captions, and voice let more people take part.
3. **Innovation.** AR, VR, AI, and connected devices succeed only if people can operate them.
4. **Economic effect.** Usable products are adopted and kept.
5. **Ethics.** The design respects privacy, avoids needless bias, and supports well-being.

1.7 Accessibility

Accessibility is the set of design choices that keep a system usable by people with different physical, sensory, and cognitive abilities. Five practices from this unit:

Contrast

Text and icons must separate from the background. Black text on white remains readable in sunlight; pale grey on white does not.

Readable type

A plain face, a comfortable size, and open spacing. A school page in a large sans-serif such as Calibri is easier than a cramped decorative face.

Colour-blind friendly

Never use colour as the only signal. A route marked only in red and green fails; the same route with a label or a pattern still works.

Inclusive navigation

Offer more than one path. Keyboard order, visible focus, and screen-reader names let a person who cannot use a mouse move with Tab.

Multimodal presentation

Say the same fact in sight, sound, or touch. Captions serve a deaf student. A vibration serves a user who cannot look at the screen.

$$\text{contrast} = \frac{L_{\text{lighter}} + 0.05}{L_{\text{darker}} + 0.05}$$

Relative luminance L is a weighted brightness from 0 to 1. Designers treat about $4.5 : 1$ as the usual minimum for body text and about $3 : 1$ for large text. The formula is the working rule behind the textbook's demand for high contrast.



Figure 1.3. High contrast on the left, a failing pair in the middle, and colour plus a word and a shape on the right so meaning survives if colour is hard to see.

1.8 Need analysis

Need analysis finds the gap between the present situation and the desired one, before any polished screen is drawn. Designers collect evidence by interview, survey, observation, and questionnaire, then design for the goals, tasks, abilities, and preferences they actually heard.

1 · USERS
Who will operate it, and who else is affected?

2 · COLLECT
Interview, watch, and survey. Do not guess.

3 · ANALYSE
Name problems, goals, and requirements.

4 · PRIORITISE
Decide what must ship first.

5 · DESIGN
Propose an interface that meets those needs.

WORKED CASE · MOBILE BANKING

Interviews and observation might show: fingerprint or face login, simple transfers and bill pay, a fast balance and history, large targets for older customers, and both English and a local language. The interface is then built from that list, not from the designer's taste alone.

1.9 Problems, and the methods that answer them

Problem	What the user feels	A direct improvement
Interface complexity	Too many choices; checkout is hidden	Simpler navigation; one clear primary action
Poor feedback	A form freezes and never says whether it sent	A visible result: “Payment successful”
Inconsistency	Two different icons both mean search	One symbol and one placement for one action
Accessibility gaps	No captions, poor contrast, no screen reader	Captions, contrast, keyboard and reader support
Compatibility	Fine on a desktop, broken on a phone	Test across browsers, sizes, and older devices
Contextual neglect	No dark mode; unusable at night	Adapt to light, screen size, and slow networks
High cognitive load	Too many codes and steps held in memory	Shorter flows; show the next step on screen
Lack of testing	Failures appear only after release	Usability tests on realistic devices
Navigation barriers	A needed form is buried in menus	Plain labels and a shallow path
Trust deficiency	No security cue, so people hesitate to pay	Clear confirmation and honest security signals

User-centred design (UCD) is the habit that ties the fixes together: start from user goals, prototype, test with real people, and change the design. Multimodal choices (touch, voice, gesture) and repeated testing after release keep the product honest.

1.10 UI, UX, wireframes, prototypes

UI components

Buttons, menus, icons, text fields, images, navigation bars, checkboxes, radio buttons, and sliders. UI is the visual and interactive surface.

UX

The whole feeling of the task. A food app that opens quickly, searches cleanly, pays safely, and tracks the order has a strong UX even before its colours are admired.

A **wireframe** is the blueprint of layout and flow, drawn before final styling. A **low-fidelity** wireframe is a rough monochrome sketch of boxes and labels, fast to change. A **high-fidelity** prototype looks near the finished product: real type, colour, images, and often clicks. A **prototype** is an early interactive model given to target users so the team can learn, then revise, before full development.

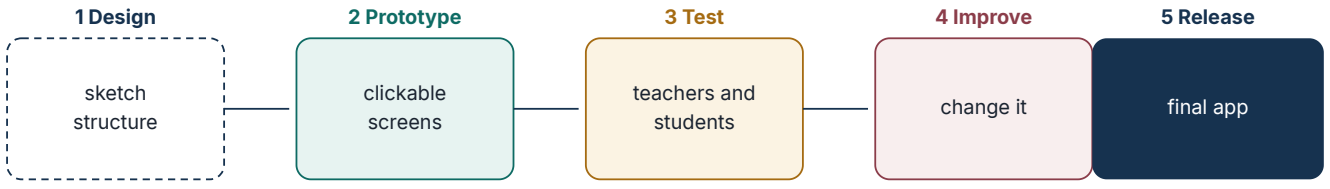


Figure 1.4. Prototype path for a student attendance app: design the idea, let teachers and students try it, revise from their comments, then build the released app.

1.11 Figma, in the order a student should work

Figma is a browser tool for shared interface design and interactive prototypes. For a low-fidelity pass, ignore polish and draw structure.

1. Create or open a file at the Figma site.
2. Press F, or choose the Frame tool, and drag a frame. A phone preset such as an iPhone frame is a sensible start. Make two frames when a flow needs two screens.
3. Use the right-hand panel for width, height, alignment, padding, fill, and corner radius.
4. Put text, rectangles, and simple shapes inside the frame. Duplicate a frame to grow the flow.
5. For the “E-Tutorial of HCI” exercise: a centred title in a plain sans-serif (Inter or Roboto, about 24 pt), a sign-up button with clearly rounded corners and a strong blue, and generous padding on a white ground.
6. Profile screen: name, batch, role, and buttons whose shape stays consistent while only the label changes (Sign up, Sign in, Sign out).
7. Switch to Prototype, link the sign-up button to the next frame, and play the flow in Presentation view.

CLASS BUILD · ATTENDANCE

Low fidelity on paper: login, class choice, mark attendance, confirmation. Rectangles and lines only. High fidelity in Figma: colour (green present, red absent), real labels, a navigation bar. Prototype links: login → class → attendance → confirmation.

1.12 Testing and evaluation

Testing asks whether the product works and where people struggle. **Evaluation** judges quality: usability, accessibility, and satisfaction, during development and after release. Tools named in the unit include the System Usability Scale (SUS) and the Net Promoter Score (NPS).

$$\text{SUS} = 2.5 \sum_{i=1}^{10} s_i$$

Ten statements, each scored 1 to 5. For odd items $s_i = \text{score} - 1$. For even items $s_i = 5 - \text{score}$. The total runs from 0 to 100. A higher score means people found the system easier.

$$\text{NPS} = \%\ \text{promoters} - \%\ \text{detractors}$$

On a 0–10 “would you recommend” question, 9–10 are promoters, 7–8 are passive and ignored in the subtraction, and 0–6 are detractors. NPS runs from -100 to $+100$.

Method	When and why
User-centred evaluation	Real users reveal preference and behaviour
Formative study	Early and repeated, while the design can still change
Summative evaluation	At the end, to measure effectiveness against the goal
Expert review	A specialist checks the design against known rules
Surveys and interviews	Opinions and scores, including SUS and NPS
Accessibility evaluation	Can people with disabilities complete the same tasks?
Usability and task tests	Watch a realistic task; record time, success, and errors
Automated audits	Tools flag contrast failures, broken links, and code errors
Keyboard and screen-reader walkthrough	Navigate with assistive technology only
Prototype, A/B, compatibility, load	Try the mock-up; compare two versions; try devices and browsers; simulate many users at once

1.13 Answers worth learning

What is accessibility in HCI?

It is designing so that people with different bodies, senses, and ways of thinking can perceive, operate, and understand the system. Contrast, readable type, non-colour cues, keyboard and reader support, and more than one sensory channel are the practical tests.

Low-fidelity and high-fidelity wireframes

Low fidelity is a fast structural sketch: placement of buttons and text, no colour. High fidelity is a detailed, often clickable design close to the final look. Use the first to decide flow, the second to judge appearance and interaction.

What fails when a system is not adaptable?

Compatibility and context. A layout that ignores phone width, a slow connection, or a dark room blocks the task. The textbook calls the matching problems compatibility issues and contextual neglect.

Why does consistency matter?

People predict the next click from the last one. The same icon, placement, and wording for the same action lowers memory load and errors. Inconsistency forces relearning on every screen.

UI and UX

UI is the surface: buttons, type, colour, layout. UX is the experience of completing the job, including speed, clarity, trust, and satisfaction. A beautiful button that leads nowhere is good UI work and poor UX.

Role of a prototype

It is an early model for learning, not the released product. Users try the flow, the team records confusion, and the final interface changes before expensive development.

2 · Computational Thinking and Algorithms

UNIT 2

An algorithm is a finite, ordered procedure that turns a valid input into the intended output. It can be written in structured steps before it is written in a programming language. This unit asks four questions of any algorithm: Is it correct? Is it clear? Is it efficient enough? Which data structure fits the relationships in the data?

- Check correctness with a trace table or stepwise reasoning
- Judge clarity: modularity and readability
- Estimate efficiency with Big O thinking
- Refine names, structure, and wasted work
- Tell linear structures from trees and graphs
- Trace retrieval in an array, a list, and a queue

2.1 Correctness

An algorithm is correct when every valid input produces the expected output. Two classroom methods are used.

Trace table

A paper execution. Columns hold the line, each variable, input and output, and the condition just tested. It is a dry run for debugging logic that “runs” but answers wrongly, and for validating a small block. It becomes tedious after hundreds of iterations.

Stepwise reasoning

Also called logic verification. Inspect each stage: Does it start in a valid state? Are all cases covered? Are the conditions right? If a loop is correct at the start of an iteration, is it still correct at the end? Better for structure and for large designs; weaker at catching a single arithmetic slip.

Worked trace · eligible heights

Count soldiers whose height h satisfies $1.57 \le h \le 1.91$. Heights: $1.91,\ 2.01,\ 1.87,\ 1.57,\ 1.48$.

```
SET total ← 0
FOR EACH height IN [1.91, 2.01, 1.87, 1.57, 1.48] DO
    IF height ≥ 1.57 AND height ≤ 1.91 THEN
        total ← total + 1
PRINT total
```

Step	height	$1.57 \le h \le 1.91$	total
start	—	—	0
1	1.91	true	1
2	2.01	false	1
3	1.87	true	2
4	1.57	true (boundary included)	3
5	1.48	false	3

Output is 3. The interesting checks are the two boundaries: 1.57 counts, 1.48 and 2.01 do not.

Worked reasoning · sum from 1 to n

```
SET sum ← 0
FOR i ← 1 TO n DO
    sum ← sum + i
PRINT sum
```

Start from a valid state $\textbf{sum} = 0$. Each integer from 1 through n is added once. When the loop ends, the variable holds the total. The closed form makes the edge cases obvious:

$$S(n) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Normal case: $n = 5$ gives $S = 15$. Edge case: $n = 0$ gives $S = 0$, because the loop body never runs. A negative n is outside the stated input and should be rejected before the loop.

SAME IDEA · A PASS MARK

Count scores ≥ 40 out of 100 with the same pattern as the height counter. The trace column is the condition $score \geq 40$, not a new kind of algorithm.

2.2 Clarity: modularity and readability

A clear algorithm can be read, repaired, and extended by someone who did not write it. Speed is not a substitute for that.

Modularity splits the work into functions or modules with one job each.

- **Single Responsibility Principle.** One module, one reason to change. A function that both reads input and sorts and prints is doing three jobs.
- **Loose coupling.** A change inside one module should not force unrelated modules to change. The module can be reused.
- **Abstraction.** The caller of a sort does not need to know whether the body is bubble sort or selection sort.

A single block that walks an array and divides by n is harder to test than two functions, $\textbf{calculateSum}(A, n)$ and $\textbf{findAverage}$, where the average is the sum divided by the count.

Readability is how fast another person can see the intent without running the steps.

- Names: $v = d/t$ hides the meaning; $\textbf{velocity} = \textbf{distance}/\textbf{time}$ shows it.
- Self-documentation: structure and names should carry the idea, with few extra comments.
- Simple control flow: shallow nesting, few tangled branches, one task per function.
- Consistency: one style of indentation, spacing, and naming.

2.3 Efficiency and Big O

Efficiency is time and memory as the input size n grows. Count the dominant steps, not every punctuation mark. Also watch the number of conditions (each is a decision; deep nesting and unpredictable branches cost clarity and sometimes time) and the number of repetitions (loops dominate).

$$T(n) \in O(f(n)) \quad \textbf{when} \quad T(n) \leq c\, f(n) \textbf{ for large } n$$

Big O is an upper-growth label. It compares algorithms and warns what will happen when the input is no longer a classroom example. Constants and lower terms are dropped: $3n + 10$ is $O(n)$.

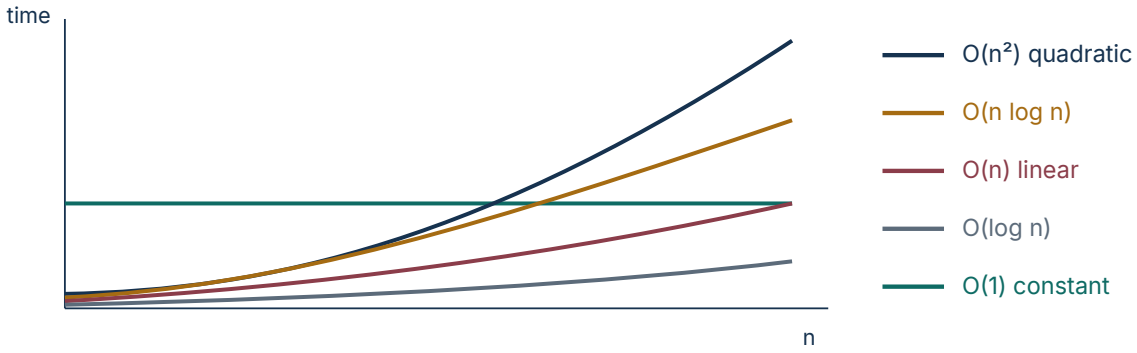


Figure 2.1. As n grows, a quadratic curve pulls away. Logarithmic and constant work stay comparatively flat. The sketch is qualitative: it shows shape, not a measured clock time.

Class	Growth	Typical source	Example
$O(1)$	Independent of n	Direct address	Read $A[0]$
$O(\log n)$	Grows very slowly	Halve the problem each step	Binary search on sorted data
$O(n)$	Proportional to n	One loop over the items	Linear search of an unsorted list
$O(n \log n)$	A little above linear	Divide and conquer	Efficient general sorting
$O(n^2)$	Square of n	Nested loops over the same data	Compare every item with every other item

A binary search of n sorted items needs about $\lfloor \log_2 n \rfloor + 1$ comparisons in the worst case. Doubling n adds only one comparison. Doubling n in a quadratic algorithm roughly quadruples the work.

Short-circuiting stops a chain of conditions as soon as the answer is known. In $h \geq 1.57 \text{ AND } h \leq 1.91$, a height of 1.20 fails the first test; the second test need not be asked. Nested conditions are a warning: they often mean a brute-force decision that is also hard to read.

Repetition grades from the unit: a single pass is a good $O(n)$; halving is an excellent $O(\log n)$; a loop inside a loop over the same collection is a poor $O(n^2)$.

2.4 Three questions before you “optimise”

1. **Does it scale?** If n grows by 10, does the time grow by about 10, or by about 100? Quadratic work often becomes unusable.
2. **What is the bottleneck?** Look at the deepest or most repeated loop. Can the same result be reached with fewer passes?
3. **Is the cost worth it?** An $O(\log n)$ algorithm that is hard to write and maintain can be the wrong choice for 50 items. A clear $O(n)$ scan may be the better engineering decision.

2.5 Refinement

Clearer

- Replace $c = t/n$ with $\text{average} = \text{totalSum}/\text{totalItems}$.
- Split one block into modules.
- Indent conditions and loops so the structure is visible.
- Comment only the part whose purpose is not obvious. Say why, not what the next line already says.

Faster or lighter

- Stop work that cannot change the answer. In bubble sort, the end of the list is settled after each pass, so the inner loop can shrink.
- Pick the algorithm that fits: linear search for a short or unsorted list; binary search when the data is sorted and large.
- Store a result instead of recomputing it.
- Match the structure to the access pattern: array for index, hash table for key, tree or graph for relationships.

2.6 Data structures

A data structure is an organised way to store data so that access, change, and processing stay efficient. If each item has a single before-and-after neighbour, the structure is **linear**. If an item connects to many others, as a hierarchy or a network, it is **non-linear**.

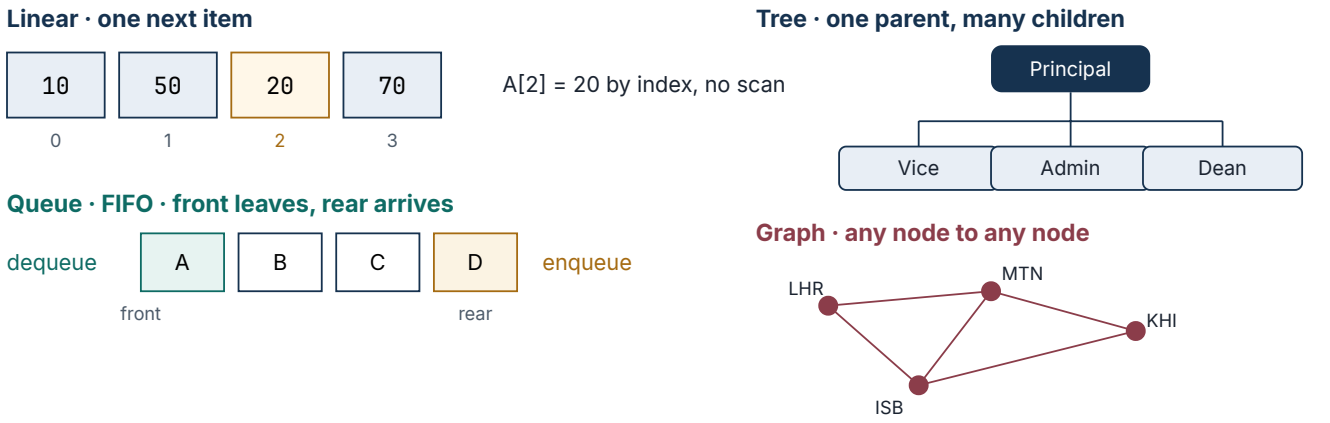


Figure 2.2. Array cells are addressed by index. A queue serves the oldest job first. A college chart is a tree. A road map is a graph: cities are vertices, roads are edges.

Structure	Idea	Use it when
Array	Fixed sequence, index from 0, direct access	A known set of similar values, such as one class of grades
Linked list	Nodes holding a value and a link to the next node	Frequent insert and delete, such as a playlist
Stack	LIFO. Push on top, pop from top. $O(1)$ at the top	Undo: the latest action reverses first
Queue	FIFO. Enqueue at the rear, dequeue at the front. Both $O(1)$ in the usual circular or linked form	Printer jobs, a fair waiting line
Tree	Root plus parent–child branches, no cycles in the simple tree	An organisation chart, a family tree
Graph	Vertices plus edges, no forced hierarchy	Friendships, GPS roads, several routes between the same places

Shared operations: **insertion**, **deletion**, **traversal** (visit each item), **search**, and **sorting** (ascending, descending, or chronological).

> **CHOOSING IN ONE LINE**
> Next/previous logic → array or list. Parent/child → tree. First come, first served → queue. Reverse the order of arrival → stack. Many-to-many connections → graph.

2.7 Tracing retrieval

Array. Index i is a direct address. With base address B and element width w,

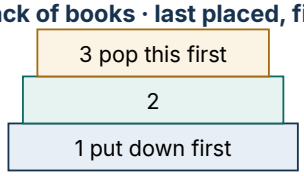
$$\text{address}(i) = B + i \cdot w$$

For $A = [10, 50, 20, 70, 90, 30]$, $A[2] = 20$. Forward traversal walks 0 to $n-1$; backward walks $n-1$ to 0. There is no search when the index is known. Cost $O(1)$.

Linked list. There is no index jump. A pointer starts at the head and follows links. To find position k, take k steps. To search for a key, compare each node until a match or NULL. In the list $56 \to 87 \to 23 \to \text{NULL}$, key 87 is found (true) and key 97 is not (false). A search is $O(n)$.

Queue. Retrieval is not "find any item". The contract is fairness: the element that arrived first leaves first. New work joins at the rear. You do not pull job C while A is still waiting, unless the structure is no longer a plain queue.

Stack of books · last placed, first removed



PUSH adds a book on top.
POP removes the current top.
Order out is 3, then 2, then 1.

A queue of students is the opposite:
join at the rear (enqueue),
leave at the front (dequeue).

Figure 2.3. Classroom model of a stack. A line of students at a printer window is the classroom model of a queue.

2.8 Answers worth learning

What is a trace table for?

It is a manual dry run that records how variables and conditions change line by line, so a logical error can be seen even when the algorithm does not crash.

Two aspects of clarity

Modularity (small single-purpose parts, loose coupling, hidden detail) and readability (names, simple flow, consistent layout, light comments).

$O(1)$ against $O(n)$

$O(1)$ does not grow with the collection: reading a known index. $O(n)$ grows in proportion: scanning an unsorted list for a name. If the list doubles, the scan's work roughly doubles; the index read does not.

Stack retrieval against queue retrieval

A stack returns the newest item (LIFO, pop). A queue returns the oldest item (FIFO, dequeue). Both restrict where removal is allowed; neither is a general search.

Array against linked list

An array stores neighbours in a fixed block and jumps by index, but inserting in the middle shifts elements. A linked list inserts and deletes by rewiring pointers, but reaching the k-th node costs k steps.

3 · Programming Fundamentals

UNIT 3

Python already knows several ways to hold a collection, and it lets the programmer wrap repeated work in a function and keep results in a file after the program stops. Variables disappear when the process ends. Files do not.

- Use lists, tuples, sets, and dictionaries
- Write functions and split a problem into them
- Separate built-in functions from user-defined ones
- Read and write text files safely

3.1 Two layers of data

Primitive values are single items: integers, floats, strings, and booleans. **Abstract** structures are built from those values and add operations: lists, tuples, sets, dictionaries, and, in the wider computer-science sense, arrays and linked lists.

	List	Tuple	Set	Dictionary
Written with	`[ ]`	`( )`	`{ }`	`{key: value}`
Order	Kept	Kept	Not a position you can index	Keys kept in insertion order (Python 3.7+)
Change later	Yes	No	Yes, add and discard	Yes
Duplicates	Allowed	Allowed	Rejected	Keys unique; a repeated key keeps the last value
Typical job	A row of items you will edit	A fixed record	Membership, union, no duplicates	Look up a value by a name

3.2 Lists

```
marks = [12, 15, 14, 3, 33]
names = ["Hassan", "Qasim", "Malika", "Asghar"]
mixed = [1, "Python", 3.14]
```

Indexing starts at 0 from the left and at -1 from the right. For five items the positive indices are $0, 1, 2, 3, 4$ and the negative indices are $-5, -4, -3, -2, -1$.

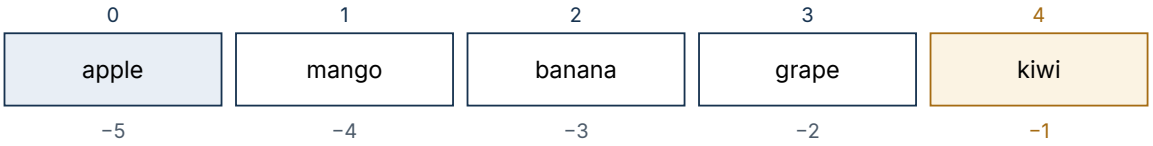


Figure 3.1. `fruits[0]` and `fruits[-5]` are both apple. `fruits[-1]` is kiwi.

`seq[start:stop:step]`

The slice begins at `start` (included) and stops before `stop`. Omitted start means the beginning; omitted stop means the end; omitted step means 1. A negative step walks backwards. Slicing returns a new list and leaves the original in place.

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
nums[2:6]    # [2, 3, 4, 5]
nums[:4]     # [0, 1, 2, 3]
nums[6:]     # [6, 7, 8, 9]
nums[::2]    # [0, 2, 4, 6, 8]
nums[::-1]   # reversed copy
```

Traversal visits every element. A `for` loop can walk values directly, or walk indices with `range(len(nums))`. A `while` loop keeps its own index and stops when the index reaches the length.

Method or function	Effect
`len(a)`	Number of items
`append(x)`	Add `x` at the end
`insert(i, x)`	Put `x` before index `i`. Index 0 inserts at the front
`remove(x)`	Delete the first matching value, or raise `ValueError`
`clear()`	Delete every item
`count(x)`, `index(x)`	How many, and the first position
`sort()`, `reverse()`	Sort in place (pass `reverse=True` for descending); reverse in place

Searching without a built-in maximum is a single scan: remember the largest value seen so far and replace it whenever a later item is greater. On $[45, 22, 89, 34, 60]$ the remembered value becomes 45, stays 45, becomes 89, and finishes at 89.

3.3 Tuples

A tuple is an ordered, immutable collection. After `point = (3, 4)`, the assignment `point[0] = 99` raises `TypeError`. You may still traverse a tuple with a loop, and if a tuple holds a mutable object such as a list, that inner object can change. Use a tuple for a record that must stay fixed, such as a coordinate or a dictionary key.

3.4 Sets

```
primes = {2, 3, 5, 7, 3}   # {2, 3, 5, 7}  — the second 3 is dropped
3 in primes                # True
```

`add` inserts one element. `remove` deletes and raises `KeyError` if the element is absent. `discard` deletes and stays quiet if it is already absent. Sets have no index, so traversal uses `for x in primes`, and the visit order is not a promise you should rely on.

$$A \cup B, \quad A \cap B, \quad A \setminus B$$

In Python: `A | B` (union, everything in either), `A & B` (intersection, only the shared elements), `A - B` (in A but not in B). This is why a set is the right tool for “unique visitors” or “students in both clubs”.

3.5 Dictionaries

A dictionary stores pairs. The key is the word you look up; the value is the definition.

```
person = {"name": "Asghar", "age": 30, "city": "Karachi"}
person["age"]            # 30
person["salary"]         # KeyError
person.get("salary")     # None
person.get("salary", 0)  # 0
"name" in person         # True, membership tests keys
person["job"] = "Engineer"
del person["city"]
```

`keys()`, `values()`, and `items()` are live views: after a new pair is inserted, a view taken earlier reflects the change. Traversal has three shapes: over keys, over values, or over `(key, value)` pairs from `items()`.

A word-frequency counter is the standard combined exercise. Split a text into words, and for each word do `freq[word] = freq.get(word, 0) + 1`. The most frequent word is `max(freq, key=freq.get)`.

Useful built-ins across these structures: `len`, `sum`, `min`, `max`, `sorted` (returns a new list), and `range`.

3.6 Functions

Duplicate code is long, brittle, and easy to update in one place while forgetting another. A function writes the steps once. Define it with `def`, then call it by name.



Figure 3.2. Name, parameters, body, and return. Arguments are the actual values supplied at the call.

Built-in

Already defined by Python: `len`, `range`, `sum`, `print`, `input`, `max`. You call them; you do not write their bodies.

User-defined

Written for this program. `check_even_odd(n)` prints Even or Odd from `n % 2 == 0`, so the test is not copied at every call site.

```
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b != 0:
        return a / b
    return "cannot divide by zero"
```

The main program only reads two numbers and prints the four calls. Decomposition is the act of cutting the calculator into those four functions instead of one long script. Each function has a single responsibility and can be tested alone.

Scope. A variable created inside a function (`result` in `add`) is local and invisible outside. A variable created outside every function is global and can be read from inside, unless the function assigns to that name, which would create a local unless `global` is declared. Prefer parameters and return values over shared globals.

3.7 Files

A text file stores characters a person can open in an editor (`.txt`, `.csv`, `.py`). A binary file stores bytes (`.jpg`, `.mp3`, `.dat`). The call is `file_object = open(path, mode)`. The default mode is `'r'`, text reading.

Mode	Action	If the file is missing
`r` / `rb`	Read text / read bytes, from the start	Error
`w` / `wb`	Write, wiping any old content	Created
`a` / `ab`	Append at the end	Created
`r+`	Read and write, file must exist	Error

After a read-mode open: `read()` returns the whole file as one string, `readline()` returns one line, `readlines()` returns a list of lines. `write(text)` stores a string; include `"\n"` when you want a new line. `close()` flushes the buffer and releases the file. Forgetting it can lose data or lock the file.

The `with` statement is a context manager. It opens the file for the block and closes it afterwards, including when an exception is raised. Think of it as a tap that shuts itself off.

```
with open("grades.txt", "w") as f:
    f.write("Alice: 88\n")
    f.write("Bob: 75\n")
with open("grades.txt", "a") as f:
    f.write("David: 81\n")
with open("grades.txt", "r") as f:
    lines = f.readlines()
```

The class average is the sum of the integer grades divided by `len(lines)`. Reading a huge file character by character after `read()` pulls the entire file into memory. Prefer `for line in new_file` when the file may be long.

An exception is an error during execution that interrupts the normal flow. Wrap file work in `try` / `except`:

- `FileNotFoundError` — the path does not exist in read mode.
- `PermissionError` — the operating system refused access.
- `IOError` — a broader reading or writing failure.

A small copy activity: open `abc.txt` for reading, open `xyz.txt` for writing, and write each character (or, better, each line) across.

3.8 Answers worth learning

List and tuple

Both are ordered and may contain duplicates and mixed types. A list can be edited in place. A tuple cannot. Choose a list for a growing collection and a tuple for a fixed record.

Why `with`?

It guarantees `close()`, so buffers are flushed and the file handle is released even if the block fails halfway.

Local and global

Local names exist only inside the function that created them. Global names are created outside functions and are visible more widely. Passing arguments and returning results keeps functions independent.

Objective checks

A list is mutable. A mixed list looks like `["apple", 42, 3.142, True]`. `fruits[-1]` is the last item. `"geek" in data` is `True` when that key exists. A set literal is `{1, 2, 3}`. A missing key used with `[]` raises `KeyError`. The unordered unindexed structure is the set. `max` returns the largest element. `clear()` empties a list. `len` counts elements. Square brackets create lists. Dictionaries store key–value pairs.

4 · Data and Analysis

UNIT 4

Data analysis examines, organises, and interprets data so that a person can decide. A school looks at results, a shop looks at sales, a hospital looks at records. Python is the bridge: it can read files and databases, arrange rows and columns, compute statistics, and draw the picture.

- Connect a Python program to a database
- Clean and reshape data, including missing values
- Draw bar charts, histograms, scatter plots, and box plots
- Compute centre and spread, and choose the right one

4.1 From source to picture



Figure 4.1. Raw sources are not yet a decision. Connectivity, a table, a summary number, and a chart are successive translations.

Sources include schools, businesses, websites, phones, sensors, files, and databases. **Data connectivity** is the communication that lets a program retrieve, store, update, and manage that data. Install the analysis libraries when they are not already present, for example `pip install pandas`.

4.2 A database connection

A **database** is an organised collection of related data. SQLite (`sqlite3`) is the lightweight engine built into Python, suitable for learning and small applications. MySQL is a server database used for larger sites. The pieces of a connection are:

Driver

Software that lets Python speak to the database system. For SQLite the driver is the `sqlite3` module.

Connection

The object that opens, holds, commits, and closes the link. Create it before any query.

Cursor

The object that executes SQL and fetches rows. It exists only after a connection.

SQL

The language of CREATE, INSERT, SELECT, UPDATE, and DELETE.

```
import sqlite3
conn = sqlite3.connect("school.db")      # creates the file if needed
cur = conn.cursor()
cur.execute("CREATE TABLE students (id INTEGER, name TEXT, marks INTEGER)")
cur.execute("INSERT INTO students VALUES (1, 'Ali', 85)")
conn.commit()                            # make the insert permanent
cur.execute("SELECT * FROM students")
for row in cur.fetchall():
    print(row)
conn.close()
```

Order that examiners like: import the library, open the connection, create the cursor, execute, commit when data changed, fetch, close. `fetchall()` returns every remaining row of the current result.

4.3 DataFrames and cleaning

A DataFrame is Pandas' two-dimensional table: rows are records, columns are attributes, like a sheet or a database table. It exists so that filtering, sorting, and aggregation have a stable shape.

```
df = pd.DataFrame(data, index=..., columns=..., dtype=...)
```

`pd.read_csv("file.csv")` and `pd.read_excel("students.xlsx", sheet_name="Sheet1")` load files. Check the path and the sheet name first. `df.head()` shows the first five rows.

New columns are ordinary arithmetic. If Math and Science are columns, `df["Total"] = df["Math"] + df["Science"]` and `df["Average"] = df["Total"] / 2` derive a result the raw marks did not state.

Missing data appears as `NaN` (not a number). `df.isnull()` is a table of True/False. `df.isnull().sum()` counts missing cells per column. `fillna` replaces them; a common choice is the column mean for a numeric mark and a label such as `"Not Assigned"` for a missing grade. `dropna()` deletes any row that still contains a gap. Cleaning matters because a mean or a chart quietly goes wrong when gaps are left in place or filled without a reason.

4.4 Organising what you will show

Table

Rows and columns for exact comparison. A student row with English, Math, Science, and an average is a table.

Frequency distribution

Counts of values falling in ranges, such as marks 50–59, 60–69, 70–79. It shows where the crowd sits when listing every student would hide the pattern.

Graph

A picture of trend, comparison, distribution, or relationship. Matplotlib (`import matplotlib.pyplot as plt`) is the library used in this unit.

Chart	Question it answers	Call
Line	How a measure changes over time	`plt.plot(years, marks, marker="o")`
Bar	Compare separate categories	`plt.bar(names, marks)`
Histogram	Distribution of a continuous measure; bars touch because bins are adjacent ranges	`plt.hist(scores, bins=...)`
Scatter	Relationship of two numeric variables	`plt.scatter(x, y)`
Box plot	Centre, spread, and extreme values in one picture	`plt.boxplot(data)`

A bar chart of subjects leaves gaps between bars because the categories are separate. A histogram of scores does not: the axis is one continuous scale cut into bins. A line of yearly percentages shows a

trend. A scatter of rainfall against crop yield asks whether the two move together. Always label axes, give a title, and call `plt.figure()` when you want a fresh canvas, then `plt.show()`.

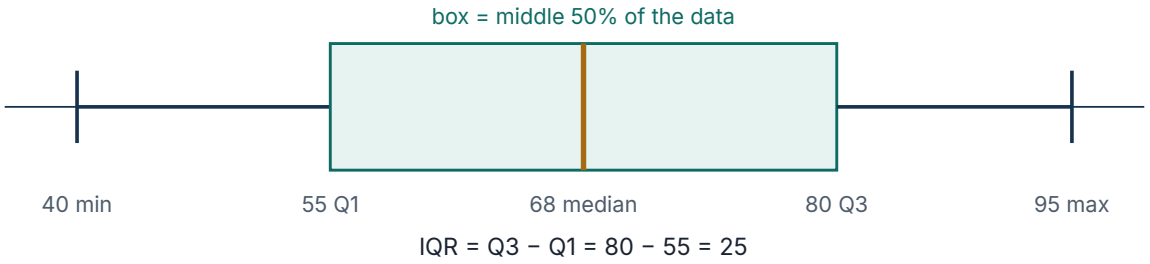


Figure 4.2. A box plot reports five numbers: minimum, first quartile Q_1, median Q_2, third quartile Q_3, and maximum. The box is the middle half. Whiskers run toward the extremes.

4.5 Centre

A measure of central tendency is one number that stands for a whole set.

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Mean. Add every value and divide by how many there are. Scores 70, 80, and 90 give $\bar{x} = (70 + 80 + 90)/3 = 80$. The mean uses every value, so one extreme salary or one very low mark pulls it.

Median. Sort the values. If the count is odd, take the middle. If the count is even, average the two middle values. For $35, 10, 15, 70, 95$, the ordered list is $10, 15, 35, 70, 95$, so the median is 35. A few very high salaries do not move the median the way they move the mean. That is why pay in a company with a handful of huge salaries is often summarised by the median.

Mode. The value that appears most often. Daily bottle sales $20, 28, 20, 30, 20, 25, 20$ have mode 20. A set may have one mode, more than one, or none. Favourite fruit in a class is a mode question, because the data are categories, not amounts you should average.

Situation	Prefer	Why
Test scores without wild outliers	Mean	Every script contributes
Salaries with a few extreme highs	Median	The middle person is a fairer picture
Most common choice	Mode	Categories have no numeric middle

4.6 Spread

$$\text{Range} = \max x - \min x$$

Notebook sales 120, 150, and 180 have range $180 - 120 = 60$. Range is quick and brittle: one unusual day sets it.

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2, \qquad \sigma = \sqrt{\sigma^2}$$

Variance is the average squared distance from the mean. Squaring stops negative deviations from cancelling positive ones. **Standard deviation** σ puts that distance back into the original units. This unit uses the $1/n$ form.

Worked low-variance day: sales $100, 102, 101, 99, 98$. The mean is $500/5 = 100$. Squared deviations: $0, 4, 1, 1, 4$, sum 10. Variance $10/5 = 2$, so $\sigma = \sqrt{2} \approx 1.41$ bottles. The shop's trade is steady.

Worked high-variance day: $50, 80, 100, 130, 140$. The mean is again 100. Squared deviations: $2500, 400, 0, 900, 1600$, sum 5400. Variance $5400/5 = 1080$, so $\sigma = \sqrt{1080} \approx 32.9$. Same centre, very different reliability.

A **deviation** is $x_i - \bar{x}$. Positive means above the mean, negative means below, zero means equal. If average sales are 100, a day of 105 deviates by $+5$ and a day of 95 by -5. Minimum and maximum are simply the smallest and largest values: in $70, 80, 90, 60, 75$, min is 60 and max is 90.

> **WHY THE MEAN CAN MISLEAD**
> Five students score 50, 52, 48, 51, and 99. The mean is 60, which suggests a comfortable pass. The median is 51, which shows that a typical student is near the pass line and one high score is doing the lifting. Report both, and show a chart.

4.7 Answers worth learning

Data analysis, in one example

It is examining organised data to support a decision. A school compares subject means and a histogram of marks to see whether the weakness is a few students or the whole class.

Why Python

The language is readable, and libraries cover files, SQLite, tables (Pandas), and charts (Matplotlib), so one program can collect, clean, summarise, and present.

Connection object and cursor

The connection opens and closes the link and commits changes. The cursor runs SQL and fetches rows. You cannot have a useful cursor before a connection.

Objective checks

Analysis interprets data. The library in this unit is Pandas. A compiler is not a data source. The connector is an interface between program and database. Importing the library comes before execute, fetch, or close. A DataFrame is a two-dimensional table. A new figure starts with `plt.figure()`. Trends over time suit a line graph. A histogram shows the distribution of continuous data. The mean is the centre most pulled by extremes. CSV files are read with `read_csv`. The gaps are filled from the unit's own sentences: files and databases; connectivity; SQLite; import.

5 · Applications and Impacts of Computing

UNIT 5

Modern systems learn patterns, and people share work across networks. This unit holds both facts together: a neural network is a mathematical model, and a collaboration tool is only fit for a college when identity, permission, privacy, and fair access are designed in.

- Explain neural networks and deep learning
- Name where those models are used
- Describe secure collaboration and data protection
- Match threats to mitigations
- Distinguish equity from equal access

5.1 Three nested ideas

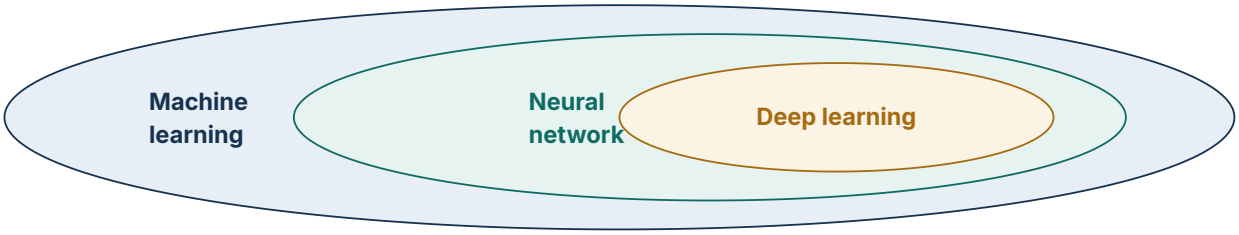


Figure 5.1. A model learns from data and then predicts. A neural network is one kind of model. Deep learning is the method of training a neural network that has many hidden layers on a large set of examples.

Training has an order. The model is given examples, it adjusts internal numbers until patterns reduce error, and only then is it asked to decide. Face unlock is the classroom picture: features of a face are compared with a stored pattern, and the phone opens only on a match. The neural network is the model. Deep learning is the method that trained it on many faces.

The word **deep** means that data passes through several layers. Early layers pick up simple pieces, such as edges in a photo. Later layers combine those pieces into eyes, then a face, then a person. A photo gallery that finds “car” or “flower” is doing that kind of layered recognition.

5.2 What one neuron computes

A neuron, or node, is a small processing unit. It receives inputs, weights each one by importance, adds a bias so the result can shift, and passes the total through an activation function that decides how strongly the signal continues.

$$z = \sum_{i=1}^{n} w_i x_i + b, \qquad a = \sigma(z)$$

x_i is an input, w_i its weight, b the bias, z the weighted sum, and a the activation. A common smooth activation is the sigmoid $\sigma(z) = 1/(1+e^{-z})$, which squashes z into the range 0 to 1.

$$L = (y - \hat{y})^2$$

A loss compares the model’s answer $\hat{y}$ with the true answer y. Training changes weights and biases to make L smaller. The textbook’s “loss function” is this idea: a measure of how wrong the current answer is.

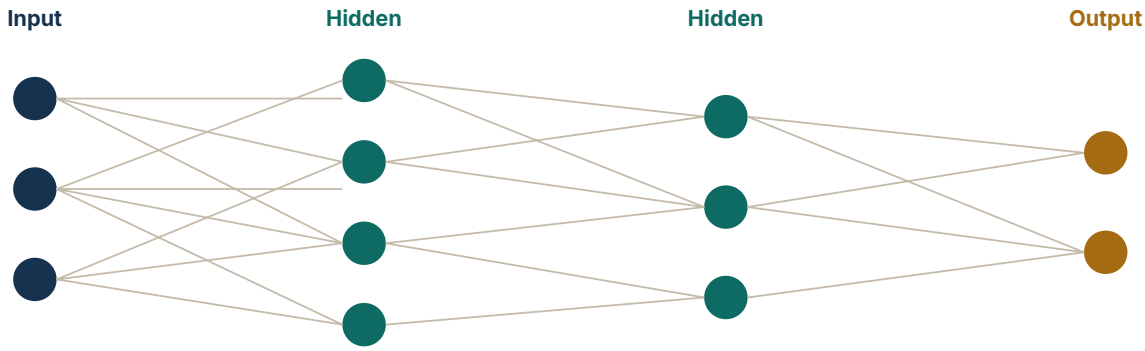


Figure 5.2. Input layer, two hidden layers, output layer. Every line is a weighted connection. Deep learning uses many hidden layers and usually needs a large collection of examples plus strong computers (often GPUs). Features can be learned from the data rather than hand-written.

Components to name in a diagram answer: input (image, sound, text, or numbers), neurons, hidden layers, weights, bias, activation, output prediction, and loss. Deep-learning components add: large data, many hidden layers, a deep network, automatic feature learning, repeated training, a learning rule that reduces error, high computing power, and large-scale uses.

5.3 Uses, including in Pakistan

The same family of models appears in image recognition (face unlock, cameras, photo search), speech recognition (a spoken alarm), natural language processing (translation, chatbots, grammar tools), healthcare images (X-ray, CT, MRI patterns that support a doctor), recommendation (the next revision video), autonomous vehicles (a pedestrian means slow down), fraud detection (a transaction that does not match past spending), and assistive tools (speech-to-text, captions, vision support).

Local applications follow the same list. Education: tutoring, automated quizzes, material matched to a weakness. Healthcare: image support where a specialist is far away. Agriculture: crop watch, weather, irrigation, yield. Banking: fraud, support, forecasting. Language: speech and translation for Urdu, Sindhi, and other regional languages. Public services: traffic and planning based on evidence. The National AI Policy 2025 is the policy context named in the unit: innovation, skills, ethical use, and digital growth. None of that works without skilled people, useful data, infrastructure, and ethical rules. The school subjects that prepare a student are programming, mathematics, data analysis, and responsible use.

5.4 Secure collaboration

Shared documents, online classes, and group files are useful only when the right people have the right access. Secure collaboration rests on five supports:

1. **Identity verification (authentication).** Prove who is asking, with a password, PIN, or one-time code. A banking app that asks for a password and then an OTP is authenticating.
2. **Access control.** After identity is known, permissions decide read, comment, or edit. Students may have view-only; a group leader may edit.
3. **Accountability.** Actions can be traced to a person, so changes are not anonymous.
4. **Data privacy.** Personal and sensitive facts — including contact numbers, passwords, marks, and records — are collected and shown only for the right purpose, to the right people.
5. **Safe communication.** Messages and files are protected from interception while they travel, not only while they sit on a disk.

Data protection keeps information safe from unauthorised access, misuse, loss, damage, or accidental deletion, and keeps it accurate and available to the people who are allowed to use it. Practical measures:

- **Encryption** turns readable plaintext into ciphertext. Symmetric encryption uses one shared key for both directions. Asymmetric encryption uses a public key to encrypt and a private key to decrypt.
- **Strong passwords** mix letter cases, digits, and symbols, and are not shared. A name or a birth date is a weak password because other people can guess it.
- **Backup and recovery.** A second copy, on another disk or a cloud store, can be restored after a crash, a deletion, or an attack.
- **Firewall and antivirus.** A firewall filters network traffic by rule. Antivirus detects and removes viruses, worms, and spyware. A lab should scan removable drives.
- **Two-factor authentication (2FA).** Two different proofs, such as a password plus an OTP. Stealing only one of them is not enough.
- **User awareness.** Do not open unexpected attachments, do not share an OTP, and do not install cracked software.

5.5 Threats and what to do

Threat	What it is	A suitable mitigation
Malware	Software meant to damage, steal, or enter: virus, worm, spyware, ransomware, Trojan. Often rides in cracked software.	Updated antivirus, licensed software, scan removable media
Phishing	A fake message that imitates a bank or a college and asks for a password or OTP	Do not follow the link; check the sender; never share an OTP; awareness
Unauthorised access	Entry without permission, helped by weak or shared passwords and unlocked devices	Strong passwords, 2FA, lock screens
Data theft / identity theft	Copying data, or using someone's identity (name, CNIC, email, bank details) as if it were yours	Encryption, least-privilege access, backups of evidence, awareness
Denial of service	A flood of fake requests so real users cannot reach a site, such as an exam portal	Firewall rules, monitoring of sudden traffic, provider-side filtering
Human error	Deleting a file, mailing the wrong department, a weak password	Training, confirmation steps, backups, limited permissions

Signs worth noticing: a slow machine, missing files, unknown programs, repeated pop-ups, failed logins from unfamiliar places, urgent messages with odd grammar or unexpected attachments, records that move by themselves, and a sudden surge of network traffic. Unsafe downloads and shared passwords create the opening even when the software is capable.

> **HYDERABAD PHISHING NOTE**
> An email that looks like the college IT office and asks for a password and an OTP is phishing. Safe actions: do not reply and do not enter the codes; contact the college through a known phone number or by walking to the office; report the message. The college will not need both secrets by email.

SUKKUR LAB CRASH

Lost mark files are a recovery problem. Regular backups on a separate disk or cloud store are the mitigation that restores them. Antivirus does not bring back a file that was never copied.

5.6 Equity and equal access

Equal access

Everyone is offered the same tools and the same chance to use them. One LMS for the whole class, so no group is left on a side channel, is equal access.

Equity

People who face an extra barrier receive the extra support that lets them participate. Captions for a deaf student, a recording for a student whose connection drops, a phone-friendly page for a student without a laptop.



Figure 5.3. Equal access gives each student the same portal. Equity adds the form of support a particular barrier requires. A fair class usually needs both.

Equity removes barriers of disability, language, distance, and cost; lets every student join the discussion; offers more than one way to learn (video, notes, audio, a worked demo); and keeps rules common so technology is not a private advantage. A rural Sindh lecture with one student who cannot hear the teacher clearly is an equity case: turn on captions. A missed live class because the connection failed is answered by a recording. Mobile-friendly pages help where the device is a phone.

Collaboration tools named in the unit: **Google Docs** (live writing and comments), **Zoom** (meetings and screen share), **Microsoft Teams** (meetings, files, class groups), **Google Classroom or another LMS** (lessons, assignments, quizzes), **WhatsApp** (short notices, only in an approved group), and **email** (formal notices and a written record).

5.7 Answers worth learning

Are neural networks and deep learning the same?

No. A neural network is the model: layers of connected nodes. Deep learning is the method of training networks that have many hidden layers on large data so they can learn complex patterns.

Why 2FA improves security

A stolen password is no longer enough. The account also demands a second, different proof, commonly an OTP sent to a registered phone.

Privacy in an online system

Marks, passwords, and identity numbers are sensitive. Privacy limits who can see them and for what purpose, which prevents misuse and keeps trust in the portal.

College with weak internet, hearing difficulty, and no personal computer

Equal access: one official class space and the same assignments for everyone, reachable on a shared lab machine or a phone. Equity: captions and a written summary for the student who cannot hear; a recording and downloadable notes for the student whose connection drops; a phone-friendly or lab-based route for the student without a computer.

Objective checks

Networks are inspired by the human brain. Deep learning relates to neural networks and is a method of machine learning. The input layer receives outside data. Authentication verifies identity. Access control decides allowed actions. Google Docs is a collaboration tool. Face unlock uses face recognition. Banks use fraud detection. Phishing tricks users into revealing secrets. Backups restore lost data.

6 · Entrepreneurship in the Digital Age

UNIT 6

An entrepreneur is a person who notices a problem, proposes a solution, and tries to turn that solution into a product or service. Entrepreneurship is the process: plan, start, test, and improve, using creativity, teamwork, communication, and calculated risk. In the digital age the tools may be a phone, a form, a social-media page, or a small site. The point of the unit is to do the sequence, not only to recite it. A paper sketch is a valid classroom tool when a lab is unavailable.

- Define entrepreneur and digital entrepreneurship with local examples
- Turn one observed problem into an idea
- Explain and compare prototype fidelities
- Define an MVP and separate it from a prototype
- Test the riskiest assumption and choose a beachhead market

6.1 The journey

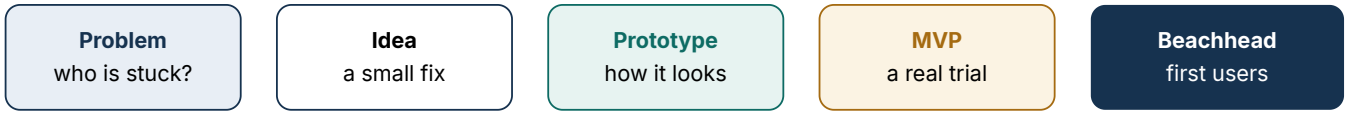


Figure 6.1. Problem, idea, prototype, minimum viable product, then a small first market. Skipping ahead to “an app for everyone” hides the riskiest guess.

Digital entrepreneurship uses those tools to start, test, and grow an idea quickly and cheaply. A WhatsApp catalogue, a Google Form, or a Facebook page can be enough for a first test. Failure at this stage is cheap, which is the reason to test early. Local starting points from the unit: fee-notice reminders for parents of first-year students; a photo catalogue for a shop that sells Ajrak, Sindhi caps, or clothes; a test-date reminder for Grade XI–XII; a canteen pre-order; a college-van update for one route; a single place for scattered exam notes.

Find the idea by watching home, college, and market. Ask what goes wrong, who suffers, how often, what people do today, and whether a simple digital step would help. Prefer a narrow problem. “Fix education” cannot be tested this term. “Remind Grade XII about practical-file day” can.

Move	Action	Example
Observe	Notice a repeated annoyance	Practical files are forgotten
Talk	Ask the people who face it	Five classmates: how do you remember?
Select	Keep one narrow problem	Practical-file reminder
Suggest	A simple paper or digital fix	One form or three screens
Test early	Show it before building the final thing	Would you actually use this?

6.2 Prototypes

A prototype is an early model of a product, service, site, or app, made so that people can see the idea and comment. It is not the finished product. It can be paper, cardboard, PowerPoint, Canva, Figma, or a sample service chart. It must be clear enough to react to. It does not need to be beautiful.

Fidelity means closeness to the final product.

Type	What it is	Best for	Example
Low-fidelity	Rough and fast, usually paper, no final colours	The first conversation about flow	Three hand-drawn homework screens
Mid-fidelity	Cleaner digital screens and navigation, not final art or live data	Testing whether people can move between screens	Clickable PowerPoint or Figma frames
High-fidelity	Realistic colour, sample data, some working parts	A later check, after the idea has already survived a rough test	A small working order form

Start low. Building a full app before anyone has confirmed the need is the common student mistake.

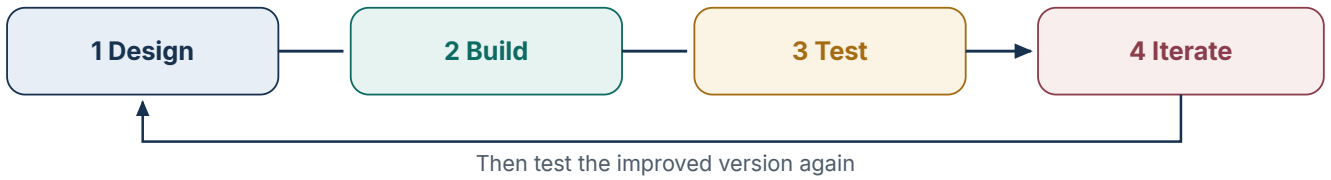


Figure 6.2. Design the flow, build a visible sample, test it with users who are not coached, then change the sample. Iteration is that change. Negative feedback means the prototype is doing its job.

In **design**, decide what must be shown: for a homework helper, an add screen, a list screen, and a reminder. In **build**, make that visible with whatever tool is at hand. In **test**, stay quiet and watch. Useful questions: What do you think this does? What is confusing? What is missing? Would you use it, and why not? In **iterate**, change the design from repeated comments. One unusual request can wait. A button five people cannot find cannot.

> **CLASSBUDDY, CONDENSED**
> Ten students are asked how often they forget a task. Suppose 7 of 10 forget at least one task a week, several want a reminder before leaving home, and some will not use a complicated app. The paper prototype then uses a large Add button, a morning reminder, and plain labels. Five classmates try to add one homework item. If they cannot find Add, enlarge it and centre it. If they want a subject and a due date, add those fields. If they need the whole week, add a weekly view. Test the second drawing on two or three new people.

6.3 The minimum viable product

An MVP is the smallest *working* version that lets real users test the main promise. Minimum means extra features are left out. Viable means it actually solves the core problem. A drawing is not an MVP. A complete canteen app with accounts, payments, tracking, ratings, and discounts is far beyond an MVP. A Google Form, or a paper register, on which students choose a snack before break can be the MVP.

	Prototype	MVP
Purpose	Show how the idea may look or flow	Test whether people will use a solution
Working?	Maybe not; a sketch is allowed	Must work in a basic way
Users	A few people comment on the design	Early real users produce real use
Cost	Very low	Still low, but some making is required
Main question	Is the idea understandable?	Will users actually use it?
Canteen example	Paper screens of menu, name, confirmation	A form used for real orders for three days
Van example	A drawing of a tracking screen	A trial WhatsApp group for one route

An MVP plan states: the problem, the first users, the one essential feature, the basic steps or screens, the low-cost tool, how early users will hear about it, how feedback will be collected, and how the team will improve. For the canteen case the feature list is menu, name, class, item, and quantity. Cost is a free form or printed slips. Marketing is a class announcement and a WhatsApp message. The test lasts three days. Feedback asks whether time was saved and whether the student would use it again. A likely improvement, once staff are heard, is to keep only five items, close orders about twenty minutes before break, and open a separate collection line.

6.4 The riskiest assumption

An assumption is a belief not yet tested. The riskiest one is the belief which, if false, collapses the idea. Test it first.

Idea	Risky belief	Early test
Online tuition	Students will pay	Ask 20 students and offer one trial class
Canteen pre-order	Students will order before break	Run the form for a few days
Ajrak catalogue	Customers will order on WhatsApp	Share with about 30 existing customers and count orders
Van updates	Parents will check daily	One route, one trial group

6.5 Beachhead market

A beachhead market is a small, specific, reachable group with a sharp need, willing to try the product first. “All students in Pakistan” and “all canteens in the city” are too broad. Better: Grade XII board students in one college; one college canteen at break; existing customers of one shop; parents on one van route. Focus is not a permanent limit. It is how the team learns before expanding.

Testing an MVP should be planned: how many users, how long, which counts, which comments. Counts show trial and repeat intent (for example, 32 of 40 tried the form, 30 of 40 would use it again). Comments explain confusion (the closing time was unclear; class name was forgotten). Act on repeated problems first.

6.6 Ethics, and a project sheet

Collect only what the test needs. Do not share phone numbers, photos, or private details without permission. Do not invent results or promise features the MVP does not have. Design for weak internet, small screens, low confidence with apps, and more than one language: large buttons, simple Urdu or Sindhi labels, and low data use are part of the product, not a decoration.

Project sheet

1. Problem statement, in one local sentence.
2. Target users.
3. Business idea.
4. Low-fidelity prototype: screens and flow.
5. Feedback from at least three users.
6. The change that feedback caused.
7. MVP: essential feature, cost, three-day-style test.
8. Riskiest assumption and how it will be tested.
9. Beachhead market and how those people will hear about it.

What a strong project shows

- The problem is local and specific.
- The prototype has a visible flow.
- Feedback was collected and is quoted, not invented.
- Changes are tied to that feedback.
- The MVP is a small working trial, not a fantasy feature list.
- The presentation can walk from problem to beachhead without changing the story.

Ideas that fit a classroom: homework or practical-file reminders, a college notice organiser, canteen pre-order, a shop catalogue, van updates, lost and found, a notes folder, library availability, a tuition-class finder, or an order form for handmade crafts.

6.7 Scenario skeletons

Forgotten practical files

Problem: the date is announced once, so absent or distracted students miss it. Idea: a subject-wise reminder with the item to bring. First prototype: low-fidelity paper screens (today's list, add item, reminder). Feedback question: "What would you need to see the night before?"

Local craft shop

Problem: customers want to see Ajrak and handmade pieces before travelling. Idea: a photo catalogue with a simple order note. First prototype: paper or a mid-fidelity Canva page of eight items. Feedback question: "Is the price and the cloth clear enough to ask for one?"

College van

Problem: students and parents do not know whether the van is late or cancelled. Idea: one daily status message. First prototype: a paper of the message format. The MVP, not the prototype, is a trial group for a single route. Feedback question: "Did this change whether you waited at the stop?"

6.8 Answers worth learning

Entrepreneur, in simple words

A person who finds a problem and works to turn a solution into something people can use, starting small and improving it after evidence.

Digital entrepreneurship

Using phones, forms, social media, sites, or simple apps to test and deliver that solution. The tool can be ordinary. The test must be real.

Why prototype before the final product?

Users reveal missing steps and confusing labels while change is still cheap. A polished build that nobody needs wastes the term.

Iteration

Improving the prototype or MVP after feedback, then testing again. The cycle is design, build, test, iterate.

Two differences: prototype and MVP

A prototype may be a non-working sketch whose question is “Do people understand this?”. An MVP must work in a basic way, and its question is “Will people use this?”.

Beachhead example

For a study-notes organiser, the beachhead is Grade XII students preparing for board exams in one college, not every student in the country.

Objective checks

An entrepreneur identifies problems and creates solutions. Digital entrepreneurship uses digital tools. A prototype is for testing and improving an idea. A paper sketch is low fidelity. The cycle is design, build, test, iterate. An MVP keeps only essential working features. The riskiest assumption is tested as early as possible. A beachhead is a small focused group of first users. Feedback shows what to improve. Paper and pencil are legitimate low-resource tools. Fill the blanks with: prototype; iteration; essential (or important); assumption; feedback.

Quick key

REVISION

Use this page after you have tried the questions, not before.

Unit 1

Main goal of HCI: make systems user-friendly. Voice commands: voice interaction. Buttons, menus, and layouts: interface design. Wireframing: designing layout. A prototype: an early interactive model. Weak feedback: users feel confused. Harder to use: inconsistent design. High cognitive load: frustration and errors. Accessibility testing: usability for people with disabilities. Usability testing: find problems before release.

Blanks: complex technology; keyboard and mouse (or touch); colour plus labels or symbols; interactive; experience (or satisfaction).

Unit 2

Paper execution: trace table. Stepwise reasoning is also called logic verification. Breaking into independent parts: modularity. One task per module: Single Responsibility Principle. $\textbf{velocity} = \textbf{distance}/\textbf{time}$: naming, and it is also self-documenting. Efficiency: time and space. Short-circuiting: stop as soon as the result is known. Linked-list element: node. Graph edges: connections. Adding at the rear: enqueue.

Blanks: Single Responsibility Principle; time and space; Big O notation; stack; edges.

Unit 3

Lists are mutable. Mixed list: square brackets. Last fruit: cherry. Key membership: True. Set: curly braces of values. Missing key: KeyError. Unordered and unindexed: set. Largest element: `max`. Automatic close: the `with` statement. Remove all list items: `clear()`.

Blanks: square; immutable; key–value; `with`; `len`.

Unit 4

Interpret data. Pandas. A compiler is not a source. Connector: interface. First step: import the library. DataFrame: two-dimensional table. New figure: `plt.figure()`. Trend: line graph. Histogram: distribution of continuous data. Most affected by extremes: mean.

Blanks: databases; connectivity; SQLite; import; `read_csv`.

Unit 5

Human brain. Deep learning: both a neural-network method and a machine-learning method. Input layer. Authentication. Access control. Google Docs. Face recognition. Fraud detection. Phishing. Regular backups.

Blanks: data; deep; tutoring (or personalised learning); responsible (or controlled); needs.

Unit 6

Identifies problems and creates solutions. Digital tools. Test and improve the idea. Low-fidelity prototype. Design–build–test–iterate. Only essential working features. As early as possible. A small focused first group. Feedback shows what to improve. Paper and pencil.

Blanks: prototype; iteration; essential; assumption; feedback.

Formula card

$$\text{contrast} = \frac{L_1 + 0.05}{L_2 + 0.05}$$

Lighter over darker. Body text is usually aimed at $4.5:1$ or more.

$$\text{SUS} = 2.5 \sum s_i, \qquad \text{NPS} = \%P - \%D$$

SUS items are rescaled to 0–4 before the 2.5. NPS ignores passives.

$$S(n) = \frac{n(n+1)}{2}, \qquad T(n) = O(f(n))$$

Closed sum, and the growth label used to compare algorithms.

$$\text{address}(i) = B + iw$$

Why an array index is $O(1)$ when the width w is constant.

$$\bar{x} = \frac{1}{n}\sum x_i, \quad \sigma^2 = \frac{1}{n}\sum (x_i - \bar{x})^2, \quad \sigma = \sqrt{\sigma^2}$$

Mean, population-style variance as used in the unit, and standard deviation.

$$\text{IQR} = Q_3 - Q_1, \qquad z = \sum w_i x_i + b, \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Box-plot spread, a neuron's weighted input, and the sigmoid activation.

ONE-SENTENCE CLOSE

Design for the person in front of the screen, prove an algorithm with a trace before you trust it, keep Python data in the structure that matches the question, summarise numbers with both a centre and a spread, protect collaboration with identity and backups, and test a small local product with a small local group before calling it a business.